

# Symbolic Planning for a Free-Climbing Orbital Inspection Robot

Assignment D6-V1: Autonomous Orbital Maintenance Platform - Initial Inspection and Mapping

Joel TOPULLI (8663382)

July 22, 2026

## Abstract

This report documents the design, modelling rationale, and evaluation of two PDDL formulations of an autonomous orbital inspection mission: a discrete classical PDDL model (Q1) and an extended PDDL+ model (Q2) that introduces continuous processes and autonomous events. A single free-climbing robot must traverse a graph of handrail locations on an orbital platform, inspect external hardware components with the correct sensing modality, manage a bounded onboard data buffer, and periodically dock to upload collected data, all while respecting a battery budget that is replenished by solar exposure. We describe the domain, the two problem instances used to stress-test each model (a permissive baseline and an adversarial instance designed to force recharging, storage management, and orbital-timing decisions), the reasoning behind every modelling decision, and a full walkthrough of the plans produced by the ENHSP-20 planner. We close with a discussion of the model's limitations and directions for extending it toward a maintenance-planning domain.

## 1 Domain and Planning Problem

### 1.1 Scenario

The scenario models a single autonomous robot that free-climbs along the handrails of an orbital platform (in our case we have considered the International Space Station) in order to visually or thermally inspect external hardware - solar panels, antennas, radiators, and similar components. The robot starts and ends its mission at a fixed *docking* location, from which collected inspection data is uploaded to ground control. Between dock visits the robot must:

1. Traverse a graph of reachable locations
2. Decide, at each location it reaches, whether to inspect a component present there or to move on
3. Select a sensor that matches the component's required inspection modality
4. Manage two coupled bounded resources: battery charge and onboard data storage during the whole mission
5. Return to the dock and upload any buffered data before the mission can be declared complete

Two independent PDDL formulations of this scenario were produced for this study:

- **Q1 - Classic PDDL** (`classic_pddl/`): a STRIPS-style model with numeric fluents but only instantaneous, discrete actions.
- **Q2 - PDDL+** (`advanced_pddl/`): the same scenario re-modelled with durative "activity" which uses a start action that enables a continuous process which gets interrupted by a closing event. Furthermore we embed the mission inside an orbital cycle not only modeling day and night behavior but it also simulating position-variant communication windows.

## 1.2 Types and Objects

Both domains share the same type hierarchy:

| Type       | Role                                                                                     |
|------------|------------------------------------------------------------------------------------------|
| robot      | The single free-climbing inspection robot                                                |
| location   | A node of the handrail graph                                                             |
| docking    | Subtype of <code>location</code> serving as the upload point and the mission's start/end |
| components | External hardware to inspect (panels, antennas, radiators, ...)                          |
| sensors    | The robot's hardware required to inspect the external components                         |

Keeping these as distinct first-class PDDL types, rather than collapsing everything into a generic object, keeps action signatures self-documenting and keeps the grounded action set small: the type hierarchy lets the grounder reject an invalid substitution (e.g. a `sensor` object for a `location` parameter slot) before instantiation, rather than needing an explicit type-membership predicate evaluated inside every action's precondition list once the planner is already searching.

## 1.3 Core Predicates and Fluents (shared concepts)

Across both models the mission state is tracked through:

- **Location:** `robot_at`, `component_at`, `location_reachable` defining the graph and where things sit on it.
- **Sensing capability:** `requires_sensor` (`component` → `modality`) and `has_sensor` (`robot` → `modality`) jointly gate whether an inspection is legal.
- **Mission bookkeeping:** `checked_component` and `data_stored` record what has actually been inspected and whether its data is sitting in the buffer.
- **Movement/inspection interlock:** `needs_inspection` (Q1) / the `busy` + event-driven equivalent (Q2) prevents the robot from leaving a location before it has explicitly decided to inspect or skip.
- **Sun:** `sun_present` (static per Q1 location, or combined with dynamic `sun_exposure` in Q2) governs both energy cost of movement/inspection and the rate of charging.
- **Numeric resources:** `battery_level`, `storage_used` / `storage`, and `data_size` per component are the two budgets the planner must respect throughout the mission.

## 2 Modelling Choices

### 2.1 Q1 - Classic PDDL (`classic_pddl/domain.pddl`)

Q1 uses the requirements `:typing`, `:strips`, `:negative-preconditions`, and `:numeric-fluents` only, excluding time and processes so that the model stays within plain classical planning. The key design decisions are:

**Decoupling “reaching a location” from “inspecting a component.”** Every `move_*` action sets `needs_inspection`, and no further movement action is applicable until either `inspect_*` or `skip_inspection` clears that flag. This is the single modelling device that forces a non-trivial ordering: the robot cannot simply drive past a component without an explicit planning decision about it.

**Sun-aware movement and inspection.** Rather than a single `move` action with a conditional effect on cost, movement and inspection are each split into two mutually exclusive actions: `move_sun`/`move_without_sun` and `inspect_sun`/`inspect_without_sun`. The sun-assisted variants cost only 60%

of the equivalent battery  $((\ast \text{ (cost ?now ?go) } 6) / 10))$ , representing solar compensation of movement or inspecting.

**Component-specific inspection.** `requires_sensor` and `has_sensor`, together with a per-sensor `inspection_cost`, mean a component can only be checked by a sensor the robot actually carries and whose modality matches. Therefore if the robot is equipped with the wrong hardware, parts of the mission become unachievable silently making the problem unsolvable.

**Onboard data as a bounded numeric resource.** `data_size(c)`, `storage_used(r)`, and `storage(r)` turn inspection payloads into a "bin-packing" sub-problem layered on top of routing. The planner has to consider if the data sizes of different components can fit together in a single trip.

**Threshold-triggered charging.** `charge_start` is only applicable once `battery_level`  $\leq 5$  and the robot is in sunlight; `charge` may then be called repeatedly (each call adds a fixed 5 units, capped at 90) until `stop_charging` is chosen by the planner. Charging is therefore an explicit part of the search space, not an automatic background process (that responsibility is handed to Q2's continuous charge process instead).

**Upload as a reset action.** `upload_data` is only executable at a docking location and resets `storage_used` to zero, which is what forces at least one return to dock and makes the storage cap actually bind on multi-component missions.

## 2.2 Q2 - PDDL+ (`advanced_pddl/domain.pddl`)

Q2 keeps the same conceptual scenario but re-expresses it with `:time`, `:continuous-effects`, and `:disjunctive-preconditions` in addition to Q1's requirements, so that actions, autonomous processes, and autonomous events can interact. The rationale for each PDDL+ construct used is given below.

### 2.2.1 Actions: start/end pairs for durative activities

Every activity that previously was an instantaneous Q1 action is split into a *start action* and a matching *end event*: `move_start/move_end` and `inspect_start/inspect_end`. The start action commits the robot to the activity (setting `transit/inspecting` and a busy flag, resetting the relevant timer fluent to 0); the matching event then autonomously closes the activity once its timer fluent has reached the required duration (`travel_time/inspection_time`). This start/process/end triple is the canonical PDDL+ idiom for representing an activity that takes real elapsed time rather than a single instant.

### 2.2.2 Processes: continuous resource evolution

Five `:process` declarations continuously evolve numeric fluents for as long as their precondition holds:

- `move_clock / inspect_clock` - advance `activity_timer / inspection_timer` at rate 1, i.e. they are simply wall-clock timers gating the corresponding `_end` event.
- `drain_battery` - decreases `battery_level` at rate `movement_cost` while the robot is in transit.
- `collect_data` - while inspecting, decreases `battery_level` at rate `inspection_cost / inspection_time` and increases `storage_used` at rate `data_size / inspection_time`, i.e. both battery cost and payload size are spread evenly over the inspection's real duration rather than being paid as a lump sum.

- `charge` - increases `battery_level` whenever `sun_exposure > 0` and the robot is below `max_battery_level`, at a rate proportional to `sun_exposure`, `solar_factor`, and `max_charge_rate`. This is the one process that runs autonomously and *concurrently* with everything else - there is no explicit “start charging” action in Q2 - representing the fact that solar panels are always producing whatever power the current geometry allows.
- `orbit_tick` - advances `orbit_time` at rate 1, unconditionally; this is the master clock that all the day/night, solar-angle, and communication-window events key off.
- `transmit_data` - while uploading and the `transmit_window_open` flag is set and the robot is at a docking, decreases `storage_used` at `upload_rate` and `battery_level` at `upload_cost`.

Using continuous increase/decrease effects driven by `(* #t ...)` rather than instantaneous `assign/increase` effects is what lets the numeric fluents evolve *during* an activity, not only at its boundaries - this is the central expressive gain of PDDL+ over Q1 and is what allows, e.g., a partially-finished inspection or upload to be meaningfully interrupted by an autonomous event such as `dead` or `comm_window_ends`.

### 2.2.3 Events: autonomous state transitions

**Activity closure** `move_end`, `inspect_end`, `upload_complete` fire automatically once their duration/quantity threshold is crossed, without the planner needing to explicitly schedule them.

**Battery-critical** `dead` fires as soon as `battery_level ≤ 0`, pinning it at 0 and setting `robot_disabled`. Making this an event (rather than only a precondition failure) lets the heuristic search prune any branch that would lead into it, since a disabled robot can never satisfy the goal.

**Day/night cycle** `day_ends/night_ends` flip a 45-minute/45-minute cycle, reset `orbit_time`, and (on `night_ends`) increment `orbit_index`; `wrap_orbit_idx` resets `orbit_index` to 0 every 16 orbits (approximating one full day), which is what allows the communication window’s `orbit_index ≤ 2` gate to recur.

**Discretised solar factor** Nine `solar_rise_*/solar_fall_*/solar_peak/solar_reset_*` events step `solar_factor` through the bands  $\{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$  as `orbit_time` advances through the orbit. Since the relationship of the generated power is expressed by:  $P \times \cos(\theta)$  we separate the 45 minute solar day into segments such that the change of the angle with which the rays hit the solar panels causes an increase or decrease of 20%.

**Sun exposure** Five `update_exposure_*` events recompute `sun_exposure ∈ {0, 1, 2}` (shade / half-lit transit / full sun) whenever the robot is stationary or in transit, night or day, letting `charge` scale smoothly with how lit the robot currently is rather than only depending on a static per-location boolean as in Q1.

**Communication window** `comm_window_starts / comm_window_ends` regulate a 10-minute transmission window which opens once per orbit for the first three orbits of every day (`orbit_index ≤ 2`, `orbit_time ∈ [15, 25)`), serving as the external gate that `transmit_data` depends on.

### 2.2.4 Numeric fluents

Beyond the battery/storage fluents inherited conceptually from Q1, Q2 introduces `orbit_time`, `orbit_index`, and `solar_factor` to drive the environment clock; `sun_exposure`, `max_battery_level`, and `max_charge_rate` to drive charging; and `activity_timer / inspection_timer` as the activity

stopwatches consumed by the `_end` events. Splitting `movement_cost` (a robot-level rate) from `travel_time` (a per-edge duration) mirrors the physical separation between “how fast the motors drain the battery” and “how long this particular translation takes,” which the continuous `drain_battery` process needs in order to charge a correctly-scaled cost over a variable travel duration.

### 2.2.5 Why storage saturation is a precondition, not an event

A deliberate asymmetry in the model: battery exhaustion is modelled as an event (`dead`) because it is something that can happen to the robot *unprompted*, mid-activity, regardless of what the planner intended. Storage saturation, by contrast, is enforced as a precondition on `inspect_start` (`(<= (+ (storage_used ?r) (data_size ?c)) (storage ?r))`), because a full buffer simply makes starting a *new* inspection illegal - it does not need to interrupt one already in progress, since `collect_data`’s rate is only ever applied to an inspection the planner already checked would fit.

## 3 Problem Instances

### 3.1 Q1 - classic pddl/

| File                        | Topology                                                                                                                                                                                           | Purpose                                                                                                                                                                                                                                                                                     |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>problem_1.pddl</code> | <code>dock ↔ site_a</code> , one component ( <code>panel_1</code> , camera)                                                                                                                        | Minimal sanity check: single edge, ample battery (50) and storage (20), full sun everywhere.                                                                                                                                                                                                |
| <code>problem_2.pddl</code> | Star topology through a hub, three components ( <code>panel_1/camera</code> , <code>radiator_1/thermal</code> , <code>antenna_1/high-res</code> ), <code>radiator_site</code> permanently shadowed | Adversarial instance: storage cap (14) is smaller than the sum of all three payloads (23), and only panel+radiator (13) fit together, forcing separate uploads. Battery starts at 19, too low to finish without recharging, and the shadowed radiator branch offers no in-transit charging. |

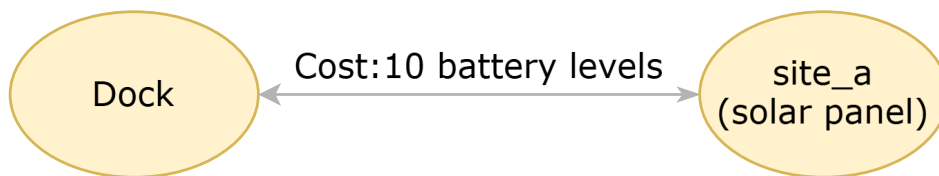


Figure 1: The environment for the simple problem of Q1

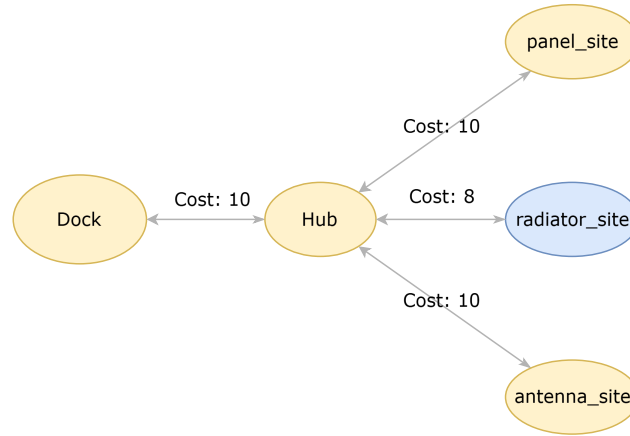


Figure 2: The environment for the hard problem of Q1

### 3.2 Q2 - advanced\_pddl/

| File                | Topology                                                                                         | Purpose                                                                                                                                                                                                                                |
|---------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| p1_sim-<br>ple.pddl | dock, loc_a, loc_b all mutually reachable, everywhere lit, daytime                               | Generous resources (battery 25/30, storage 50). Routing and battery never bind; the only real constraint is the recurring communication window - the one decision that matters is <i>when</i> to be at dock so an upload can complete. |
| p2_-<br>hard.pddl   | Forced relay: loc_b reachable only via loc_a (no dock↔loc_b shortcut); loc_b in permanent shadow | Battery starts low (18/30); loc_b must be reached and left on a single reserve of charge; the same recurring comm-window gate applies, forcing the mission to be staged across multiple daily cycles.                                  |

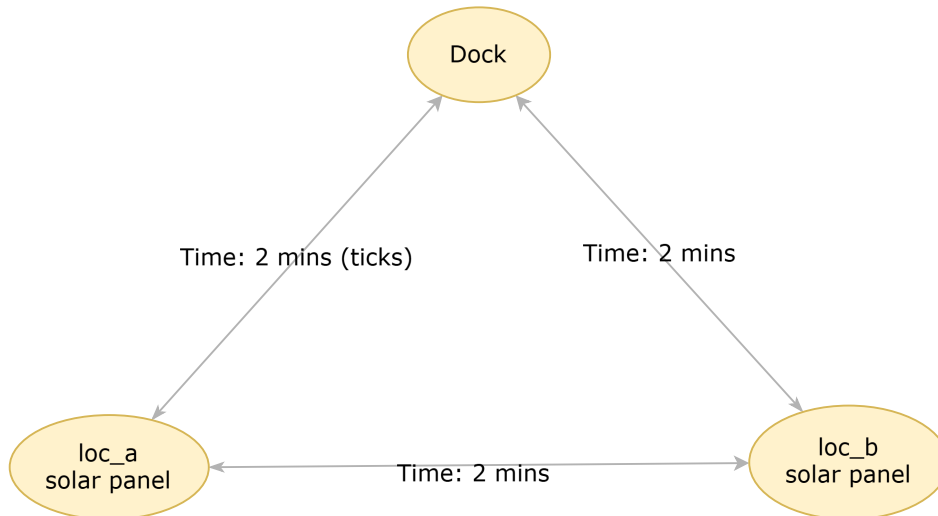


Figure 3: The environment for the simple problem of Q2

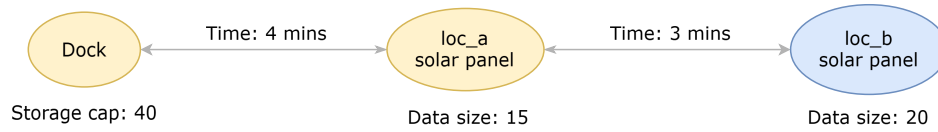


Figure 4: The environment for the hard problem of Q2

## 4 Plan Walkthrough

### 4.1 Q1 - Simple Plan (problem\_1.pddl)

```

1 0.00000: (move_sun rob1 dock site_a)
2 1.00000: (inspect_sun rob1 site_a panel_1 camera)
3 2.00000: (move_sun rob1 site_a dock)
4 3.00000: (upload_data rob1 dock)

```

Listing 1: Plan for problem\_1.pddl

With a single sunlit edge, one component, and generous battery (50) and storage (20), there is exactly one sane action sequence - move to `site_a`, inspect `panel_1` with the `camera` sensor it requires, move back, and upload. The planner is not solving a meaningful search problem here; the instance instead confirms the domain's baseline move → inspect → move → upload loop is correctly wired, and that the sun-assisted action variants are chosen in preference to their (here inapplicable, as both edges are lit) non-sun counterparts. **This behaves exactly as expected** for a baseline sanity-check instance.

### 4.2 Q1 - Hard Plan (problem\_2.pddl)

The 57-step plan (full listing in Appendix 7.2) makes **three** separate dock round-trips - one per component - each interleaved with charging at hub:

1. **Scout antenna, then charge.** The robot moves `dock`→`hub`→`antenna_site`→`hub`, skipping inspection both times, then runs a full `charge_start/charge×7/stop_charging` cycle at `hub` before committing to the shaded branch.
2. **Radiator branch (shaded).** With reserve charge banked, the robot dives into `radiator_site` - the only branch with no solar presence - inspects `radiator_1` with the `thermal_camera`, and returns to `hub`.
3. **First upload + partial recharge.** Back at `dock`, it uploads (clearing storage), then tops up its battery slightly before heading out again.
4. **Antenna branch.** A second charge cycle at `hub`, then the robot revisits `antenna_site`, this time actually inspecting `antenna_1` with `highres_camera`, and returns to `dock` for a second upload.
5. **Panel branch and final upload.** A third pass through `hub` to `panel_site` inspects `panel_1` with `camera`, a short top-up charge follows, and the robot returns to `dock` for the third and final upload, accomplishing the goal.

This is the expected behaviour given the instance design: storage (14) cannot hold all three payloads (23) simultaneously, and even the best achievable pairing (panel + radiator, 13) is not chosen - a reminder that a *valid* plan under a satisficing planner is not necessarily the most storage-efficient one. The starting battery (19) is genuinely too low to complete the mission without recharging, and the plan does recharge exactly at the points where it is structurally necessary: before the unassisted shaded leg, and before each subsequent outbound trip. Since every Q1 action is unit-cost and instantaneous, there is no idle/waiting time to account for.

However a very interesting aspect of the domain is exposed by this plan. If we trace the battery through those first steps (`move_sun` costs  $(\text{cost} \times 6)/10$ , since sun covers 40% of the draw):

- Start: battery = 19, at dock.
- Step 0: `move_sun dock → hub`: cost  $10 \times 0.6 = 6 \rightarrow$  battery 13.
- Step 2: `move_sun hub → antenna_site`: cost  $10 \times 0.6 = 6 \rightarrow$  battery 7.
- Step 4: `move_sun antenna_site → hub`: cost  $10 \times 0.6 = 6 \rightarrow$  battery 1.

So the antenna round-trip isn't about the antenna at all - the robot does nothing there (`skip_inspection` both times) and just comes straight back. Since the robot can only begin charging once its battery is already at or below 5 and there is sun present at its location it executes a battery-draining maneuver. It starts at 19 which is too high to trigger charging at all. And it can't sit still to drain battery; the only way `battery_level` decreases in this domain is via `move_sun`/`move_without_sun` or an inspection action due to the fact that there is no "wait" or "idle drain" action.

So to get battery down to the  $\leq 5$  threshold while still ending up somewhere sunlit (so `charge_start` is legal), the robot has to burn charge through movement. A single `dock → hub` hop only gets it to 13 which isn't low enough. It needs a second round trip to drain further, and it has to land back on a sunlit node (hub has `sun_present`) to be allowed to charge at all.

The robot performs this seemingly wasteful detour because it cannot go directly to the radiator site as it would leave it stranded without the possibility of executing any other actions. If we consider the direct route `dock → hub → radiator_site` with no charging stop. Starting at battery 19, the first hop (`dock → hub`) is sun-assisted, costing  $10 \times 0.6 = 6$  and leaving battery at 13. The second hop (`hub → radiator_site`) has no sun coverage, so the full cost of 8 applies, leaving battery at 5.

At this point the robot is stuck at `radiator_site` with battery 5, and every remaining action becomes inapplicable:

- **It cannot inspect:** `inspect_without_sun` requires  $(\geq (\text{battery\_level } r) (\text{inspection\_cost thermal\_camera}))$ , i.e. battery  $\geq 10$ , but only 5 remains.
- **It cannot leave:** even skipping the inspection, `move_without_sun radiator_site → hub` costs 8, which the robot cannot afford either.
- **It cannot charge on-site:** `charge_start` requires  $(\text{sun\_present } l)$ , but `radiator_site` has no such fact as it is permanently shaded.

### 4.3 Q2 - Simple Plan (`p1_simple.pddl`)

```

1 00:00 - 24:00    waiting (pre-mission, aligning to comm window)
2 24:00 - 24:08    transit dock -> loc_a
3 24:08 - 24:15    inspect panel1 @ loc_a (camera)
4 24:15 - 24:17    transit loc_a -> dock
5 24:17 - 25:24    waiting for transmit_window_open, then uploading
6 25:24 - 25:41    transit dock -> loc_b
7 25:41 - 25:44    inspect panel2 @ loc_b (spectrometer)
8 25:44 - 25:46    transit loc_b -> dock
9 25:46           final upload completes

```

Listing 2: Key milestones, `p1_simple.pddl` (durations in seconds from mission start)



The robot performs two fully sequential round trips through dock rather than one combined loop, even though nothing about battery or storage forces this separation (both are generous in this instance). Of the  $\approx 107$  s of real activity, the dominant single chunk ( $\approx 67$  s) is simply the robot parked at dock waiting for `transmit_window_open` before the first upload can finish. **This is the expected behaviour:** as designed, this instance's only binding constraint is the recurring communication window, so the plan's shape is driven entirely by *when* that window happens to be open relative to the robot's arrival, not by route or resource choices.

#### 4.4 Q2 - Hard Plan (`p2_hard.pddl`)

The hard instance's plan visits `loc_a` **three times**, and explicitly defers inspecting `panel1` on the first two visits:

1. **First pass** ( $t \approx 306$ - $310$ s). The robot travels `dock`→`loc_a`, explicitly `skip_inspection` `panel1`, and instead waits ( $\approx 1327$ s) at the sunlit `loc_a` to build up enough charge before committing to the shaded segment beyond it.
2. **Shaded segment.** It moves `loc_a`→`loc_b` on the pre-charged reserve (no in-transit or on-site top-up is possible there), waits out the remaining orbital/comm-window timing ( $\approx 1435$ s), then inspects `panel2` with the `spectro` sensor.
3. **Return, second deferral, first upload.** The robot retraces `loc_b`→`loc_a`→`dock`, skipping `panel1` again on the way through `loc_a`, and uploads `panel2`'s data plus a brief recharge at dock.
4. **Third pass - panel1 finally inspected.** Only on a *third* visit to `loc_a`, after another charge/comm-window wait ( $\approx 1369$ s), does the robot actually inspect `panel1` with `cam`, then return to dock for the second and final upload.

The transits and inspections together total only  $\approx 31$  s of real activity out of a  $4528$  s (i.e.  $\approx 75.5$ -minute) mission - over 99% of the elapsed time is the robot parked at a sunlit node waiting for `charge` and the orbit/day-night/comm-window events to bring the environment into a usable state. **This is the expected and intended behaviour of the instance:** `loc_b`'s permanent shadow means the shaded segment has to be attempted on a pre-committed battery reserve rather than a background top-up, so the robot must bank charge *before* it is needed rather than during the excursion; and the same recurring comm-window gate that dominated the simple instance forces dock arrivals to also line up with an open window, which is what pushes the mission across multiple daily/orbital cycles instead of a single continuous loop. The repeated `skip_inspection` calls at `loc_a` are a direct, visible symptom of the `needs_inspection`-style lock forcing an explicit inspect-or-skip decision every time the robot passes through a location that has not yet had its component checked.

## 5 PDDL+ vs. Classical PDDL: Summary of Differences

| Aspect                       | Q1 (classical PDDL)                                              | Q2 (PDDL+)                                                                                                                                                                |
|------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Movement/inspection duration | Instantaneous action, cost paid as a lump sum                    | *_start action + *_clock process + *_end event; cost accrues continuously over real time                                                                                  |
| Battery                      | Discrete decrease per action                                     | Continuous drain_<br>battery/collect_<br>data/transmit_data processes,<br>scaled by #t                                                                                    |
| Charging                     | Discrete charge action, +5% per call, only when battery $\leq 5$ | Continuous charge process, rate scales with sun_exposure and discretised solar_factor                                                                                     |
| Sun/shadow                   | Static per-edge/per-location boolean                             | Dynamic sun_exposure $\in \{0,1,2\}$ recomputed by events; solar_factor varies with orbital position                                                                      |
| Day/night & comms            | Not modelled                                                     | orbit_tick process + day_<br>ends/night_ends/wrap_orbit_<br>idx/comm_window_* events model a 90-minute orbit, a recurring 16-orbit day, and periodic transmission windows |
| Battery-critical condition   | Implicit (actions become inapplicable)                           | Explicit dead event disabling the robot at battery_level $\leq 0$                                                                                                         |
| Feasibility driver           | Which route/order of inspections fits storage & battery          | The above <i>plus</i> exact timing: whether activities finish and the robot reaches dock inside a recurring open comm window before running out of charge                 |
| Idle/waiting time            | None - every step is a unit-cost action                          | Dominant - both Q2 plans are 90%+ idle waiting                                                                                                                            |

The PDDL+ model is strictly more expressive but substantially more expensive to ground and search: the dominant cost in both Q2 missions is no longer distance or battery but simply waiting for the orbit to bring the comm window (and, in the hard case, sufficient charge) back around - a behaviour with no analogue in the discrete Q1 model at all.

## 6 Conclusion

The classical PDDL model (Q1) validates the core mission logic such as routing, sensor-matched inspection, bounded storage, and threshold-triggered charging against two instances of increasing difficulty, with the hard instance forcing a genuinely non-trivial interleaving of recharge cycles and separate dock trips due to a storage cap smaller than the combined payload. The PDDL+ model (Q2) preserves this same core logic while adding real-valued durations, continuous battery/storage dynamics, and an orbital day/night and communication-window cycle, at the cost of a much larger and more expensive search space. Both hard instances behave as designed: Q1's plan is shaped by resource bin-packing and recharge timing, while Q2's plan is shaped almost entirely by orbital/comm-window timing, with real robot activity accounting for well under 1% of total mission duration. The resulting models and plans provide a validated foundation that could be extended and merged with the other assignments of this call.

## 7 Appendix: Full Plans

### 7.1 Q1 - Simple Plan (problem\_1.pddl)

See Listing 1 above.

### 7.2 Q1 - Hard Plan (problem\_2.pddl)

```

1 0.00000: (move_sun rob1 dock hub)
2 1.00000: (skip_inspection rob1)
3 2.00000: (move_sun rob1 hub antenna_site)
4 3.00000: (skip_inspection rob1)
5 4.00000: (move_sun rob1 antenna_site hub)
6 5.00000: (charge_start rob1 hub)
7 6.00000: (charge rob1 hub)
8 7.00000: (charge rob1 hub)
9 8.00000: (charge rob1 hub)
10 9.00000: (charge rob1 hub)
11 10.00000: (charge rob1 hub)
12 11.00000: (charge rob1 hub)
13 12.00000: (charge rob1 hub)
14 13.00000: (stop_charging rob1)
15 14.00000: (skip_inspection rob1)
16 15.00000: (move_without_sun rob1 hub radiator_site)
17 16.00000: (inspect_without_sun rob1 radiator_site radiator_1 thermal_camera)
18 17.00000: (move_without_sun rob1 radiator_site hub)
19 18.00000: (skip_inspection rob1)
20 19.00000: (move_sun rob1 hub dock)
21 20.00000: (upload_data rob1 dock)
22 21.00000: (charge_start rob1 dock)
23 22.00000: (charge rob1 dock)
24 23.00000: (stop_charging rob1)
25 24.00000: (skip_inspection rob1)
26 25.00000: (move_sun rob1 dock hub)
27 26.00000: (charge_start rob1 hub)
28 27.00000: (charge rob1 hub)
29 28.00000: (charge rob1 hub)
30 29.00000: (charge rob1 hub)
31 30.00000: (charge rob1 hub)
32 31.00000: (stop_charging rob1)
33 32.00000: (skip_inspection rob1)
34 33.00000: (move_sun rob1 hub antenna_site)
35 34.00000: (inspect_sun rob1 antenna_site antenna_1 highres_camera)
36 35.00000: (move_sun rob1 antenna_site hub)
37 36.00000: (skip_inspection rob1)
38 37.00000: (move_sun rob1 hub dock)
39 38.00000: (upload_data rob1 dock)
40 39.00000: (charge_start rob1 dock)
41 40.00000: (charge rob1 dock)
42 41.00000: (charge rob1 dock)
43 42.00000: (charge rob1 dock)
44 43.00000: (charge rob1 dock)
45 44.00000: (stop_charging rob1)
46 45.00000: (skip_inspection rob1)
47 46.00000: (move_sun rob1 dock hub)
48 47.00000: (skip_inspection rob1)
49 48.00000: (move_sun rob1 hub panel_site)
50 49.00000: (inspect_sun rob1 panel_site panel_1 camera)
51 50.00000: (move_sun rob1 panel_site hub)
52 51.00000: (skip_inspection rob1)
53 52.00000: (charge_start rob1 hub)
54 53.00000: (charge rob1 hub)
55 54.00000: (stop_charging rob1)
56 55.00000: (move_sun rob1 hub dock)
57 56.00000: (upload_data rob1 dock)

```

Listing 3: Full 57-step plan for problem\_2.pddl

### 7.3 Q2 - Simple Plan (p1\_simple.pddl)

```

1 Trip 1 (panell1):
2   transit dock -> loc_a           (~8s)
3   inspect panell1 @ loc_a (cam)   (~7s)
4   transit loc_a -> dock           (~2s)
5   wait for comm window            (~67s)
6   start_upload @ dock
7
8 Trip 2 (panel2):
9   transit dock -> loc_b           (~17s)
10  inspect panel2 @ loc_b (spectro) (~3s)
11  transit loc_b -> dock           (~2s)
12  start_upload @ dock (final upload completes)
13
14 Total real activity: ~107s. Pre-mission wait to align with the
15 first comm window: ~1440s (24 minutes).
```

Listing 4: Milestone summary, p1\_simple.pddl

### 7.4 Q2 - Hard Plan (p2\_hard.pddl)

```

1 Pass 1 (loc_a, defer panell1):
2   wait (pre-mission)              ~306s
3   transit dock -> loc_a           ~4s
4   skip_inspection (defer panell1)
5   wait (charge before shaded leg) ~1327s
6
7 Shaded leg (panel2):
8   transit loc_a -> loc_b           ~3s
9   wait (before inspecting)        ~1435s
10  inspect panel2 @ loc_b (spectro) ~5s
11
12 Return, defer panell1 again, first upload:
13  transit loc_b -> loc_a           ~3s
14  transit loc_a -> dock           ~4s
15  skip_inspection (defer panell1, 2nd pass)
16  wait (upload panel2 + recharge)  ~59s
17
18 Final pass (panell1):
19  transit dock -> loc_a           ~4s
20  wait (comm window / charge)      ~1369s
21  inspect panell1 @ loc_a (cam)    ~4s
22  transit loc_a -> dock           ~4s
23  final upload completes
24
25 Total mission time: 4528s (~75.5 min). Real activity: ~31s (<1%).
```

Listing 5: Milestone summary, p2\_hard.pddl